A D PATEL INSTITUTE OF TECHNOLOGY

(The Charutar Vidya Mandal University)

GSIRE An ISO 9001:2015 Certified Institute

Department of Information Technology

PROJECT REPORT

ON

STUDENT CSV DATA ANALYZER

Marks, Grades, Ranks and Statistics from a CSV File

Subject: Python

Submitted by

JAYDEV H BHATT

Enrollment No.: 12402080601051

IT-A | 1C8 | 5th Semester

Academic Year: 2026-27

CERTIFICATE

This is to certify that the project report entitled “Student CSV Data Analyzer”, prepared for the subject Python, has been submitted by JAYDEV H BHATT (Enrollment No. 12402080601051), a student of the 5th Semester, Department of Information Technology, in partial fulfilment of the requirements for the degree programme. The project work has been carried out as an academic mini project and the report is found to be satisfactory.

Guide:

Name of Guide: Prof. Nayan Mali

Designation: Assistant Professor

Head of Department:

Name: DR. PRIYANG BHATT

Date: _____

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my project guide for the guidance, encouragement and feedback provided during the development of this project. I also thank the Head of the Department of Information Technology and all faculty members of A D Patel Institute of Technology for providing the academic environment and resources needed to complete this work. Finally, I am grateful to my classmates and family for their support and motivation.

JAYDEV H BHATT

ABSTRACT

Student CSV Data Analyzer is a Python mini project that automates the processing of student marks stored in a comma-separated values (CSV) file. Instead of manually calculating totals, percentages, grades and ranks, the application reads student records, validates the data and produces a consistent result summary. It also calculates class and subject statistics, lists the top-performing and failed students, shows grade distribution, supports student search and exports the processed results to CSV.

The project separates reusable processing logic in analyzer.py from two user interfaces: cli.py for a terminal menu and app.py for a browser-based Streamlit application. The program uses Python's CSV handling, lists, dictionaries, functions and sorting features. Input validation identifies missing names, non-numeric marks and marks outside the 0–100 range; invalid rows are reported and skipped. A mark below 40 in any subject causes the student to be marked as failed.

TABLE OF CONTENTS

Section	Page
Certificate and Acknowledgements	2
Abstract and Table of Contents	3
1. Introduction and Objectives	4
2. System Design and Implementation	5
3. Results, Conclusion and References	6

PROJECT OBJECTIVES

- Read student marks from a CSV file and validate each record.
- Calculate total marks, percentage, grade, pass/fail status and rank.
- Generate subject statistics, toppers, failed-student lists and grade distribution.
- Provide both a command-line interface and a Streamlit web interface using the same logic.

CHAPTER 1 – INTRODUCTION

1.1 Problem Statement

Student result processing often involves repetitive calculations in spreadsheets or by hand. This can take time and introduce mistakes in totals, percentages, grades and ranks. Incorrect or incomplete input records can also go unnoticed. The project addresses these problems by processing a structured CSV file and generating the required academic summaries automatically.

1.2 Scope and Objectives

The application is intended for small educational datasets where each student has a roll number, name and one or more subject marks. It assumes marks are out of 100 and uses 40 as the pass mark for every subject. It accepts CSV input and does not persist data in a database.

1.3 System Architecture

Component	Responsibility
students.csv	Sample input data containing student identifiers and subject marks.
analyzer.py	Reads and validates CSV data, calculates results and creates summaries.
cli.py	Provides menu-based access through a terminal.
app.py	Provides browser-based interaction through Streamlit.

1.4 Processing Pipeline

The application follows a simple pipeline: read CSV text → clean and validate rows → calculate totals, percentages, grades, status and ranks → compute class and subject statistics → display results or export them to a new CSV file. The same analysis functions are reused by both interfaces, avoiding duplicate calculation logic.

CHAPTER 2 – SYSTEM DESIGN AND IMPLEMENTATION

2.1 Technologies and Concepts

Python is used for the core logic. csv.DictReader reads rows as dictionaries and csv.DictWriter exports processed results. Lists store records and marks; dictionaries represent student records and statistics; functions separate the processing tasks. Streamlit provides the web interface, while the command-line interface offers a lightweight terminal option.

2.2 Input Validation and Grade Calculation

The CSV header must contain roll_no and name followed by subject columns. Each subject mark must be numeric and lie between 0 and 100. A missing student name, non-numeric mark or out-of-range value causes the row to be skipped with a validation message. Grades are assigned using percentage thresholds: A+ for 90 and above, A for 80–below 90, B for 70–below 80, C for 60–below 70, D for 50–below 60, E for 40–below 50 and F below 40. A student fails if any subject mark is below 40.

Percentage	**Grade**
90 and above	A+
80 to <90	A
70 to <80	B
60 to <70	C
50 to <60	D
40 to <50	E
Below 40 in any subject	F / Fail

2.3 Main Features

- Display all processed student results, including total, percentage, grade, status and rank.
- Summarise the class average, highest and lowest percentages, pass count and pass percentage.
- Calculate subject-wise average, highest mark, lowest mark and pass rate.
- Show the top five students, failed students and grade distribution; search for a student by available identifying details.
- Save processed results as a CSV file for later use.

2.4 How to Run the Project

Install the dependencies from requirements.txt in a Python environment. Run “python cli.py” for the terminal menu. To launch the browser-based interface, run “streamlit run app.py” and open the local address shown by Streamlit. The bundled students.csv is used as sample data; the web app can also accept another CSV file.

2.5 Important Implementation Logic

The result-calculation function collects a student’s subject marks, sums them to obtain the total and divides by the maximum possible marks to calculate the percentage. A grade function checks thresholds from highest to lowest. The program then sorts students by total to assign ranks. Summary functions calculate class-level metrics and iterate through each subject to derive its statistics. The project source files are provided separately with the application.

CHAPTER 3 – RESULTS AND ANALYSIS

The bundled sample file contains 30 student records and five subjects: Maths, Python, DBMS, Physics and English. The following values are the output calculated from that sample dataset.

Metric	Observed result
Students analysed	30
Class average	67.6%
Highest percentage	91.2%
Lowest percentage	41.4%
Students passed	27
Students failed	3
Pass percentage	90.0%

3.1 Subject-wise Summary

Subject	Average	Highest	Lowest	Pass rate
Maths	67.67	99	22	93.33%
Python	67.00	99	31	96.67%
DBMS	67.50	99	29	96.67%
Physics	67.77	97	36	96.67%
English	68.07	99	42	100%

3.2 Findings, Limitations and Future Scope

The highest percentage in the sample is 91.2%, and the class pass percentage is 90%. English has the highest subject average (68.07) and all 30 sample marks meet the 40-mark pass threshold. These are illustrative results from the bundled dataset, not results from a real institutional examination. The application currently supports CSV only, assumes a maximum of 100 marks and a fixed pass mark of 40, and does not store records in a database. Future improvements could include Excel import, configurable grading rules, PDF report cards, database storage and semester-wise comparisons.

3.3 Conclusion

Student CSV Data Analyzer converts raw student marks into ranked results and useful class statistics with minimal manual effort. Its modular design keeps the core processing in analyzer.py and makes it available through both a CLI and a Streamlit interface. The project demonstrates practical use of CSV processing, validation, lists, dictionaries, functions and sorting in Python.

REFERENCES

1. Python Software Foundation, Python 3 Documentation – CSV File Reading and Writing, https://docs.python.org/3/library/csv.html

2. Streamlit Documentation, https://docs.streamlit.io/

3. Student CSV Data Analyzer project source files: analyzer.py, cli.py, app.py, students.csv and README.md.